

14.5 Directed Acyclic Graphs

Directed graphs without directed cycles are encountered in many applications. Such a directed graph is often referred to as a **directed acyclic graph**, or **DAG**, for short. Applications of such graphs include the following:

- Prerequisites between courses of an academic program.
- Inheritance between classes of an object-oriented program.
- Scheduling constraints between the tasks of a project.

We will explore this latter application further in the following example:

Example 14.20: *In order to manage a large project, it is convenient to break it up into a collection of smaller tasks. The tasks, however, are rarely independent, because scheduling constraints exist between them. (For example, in a house building project, the task of ordering nails obviously precedes the task of nailing shingles to the roof deck.) Clearly, scheduling constraints cannot have circularities, because they would make the project impossible. (For example, in order to get a job you need to have work experience, but in order to get work experience you need to have a job.) The scheduling constraints impose restrictions on the order in which the tasks can be executed. Namely, if a constraint says that task a must be completed before task b is started, then a must precede b in the order of execution of the tasks. Thus, if we model a feasible set of tasks as vertices of a directed graph, and we place a directed edge from u to v whenever the task for u must be executed before the task for v , then we define a directed acyclic graph.*

14.5.1 Topological Ordering

The example above motivates the following definition. Let $\vec{G}$ be a directed graph with n vertices. A **topological ordering** of $\vec{G}$ is an ordering $v_1, \dots, v_n$ of the vertices of $\vec{G}$ such that for every edge (v_i, v_j) of $\vec{G}$, it is the case that $i < j$. That is, a topological ordering is an ordering such that any directed path in $\vec{G}$ traverses vertices in increasing order. Note that a directed graph may have more than one topological ordering. (See Figure 14.12.)

Proposition 14.21: $\vec{G}$ has a topological ordering if and only if it is acyclic.

Justification: The necessity (the “only if” part of the statement) is easy to demonstrate. Suppose $\vec{G}$ is topologically ordered. Assume, for the sake of a contradiction, that $\vec{G}$ has a cycle consisting of edges $(v_{i_0}, v_{i_1}), (v_{i_1}, v_{i_2}), \dots, (v_{i_{k-1}}, v_{i_0})$. Because of the topological ordering, we must have $i_0 < i_1 < \dots < i_{k-1} < i_0$, which is clearly impossible. Thus, $\vec{G}$ must be acyclic.

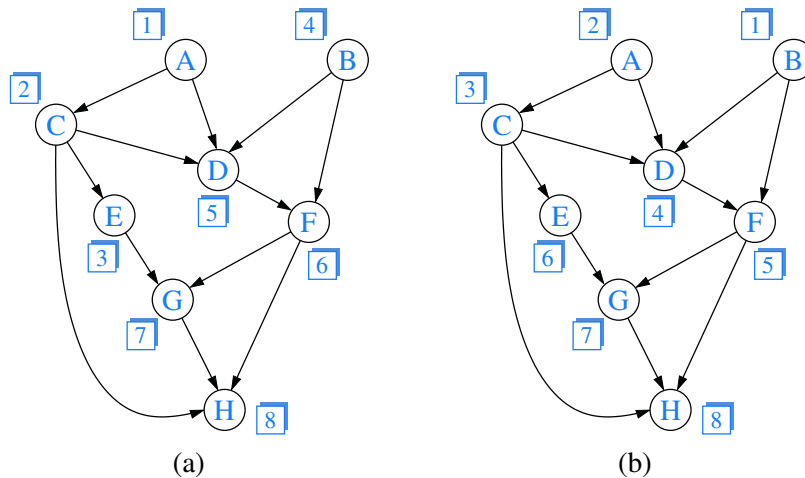


Figure 14.12: Two topological orderings of the same acyclic directed graph.

We now argue the sufficiency of the condition (the “if” part). Suppose $\vec{G}$ is acyclic. We will give an algorithmic description of how to build a topological ordering for $\vec{G}$. Since $\vec{G}$ is acyclic, $\vec{G}$ must have a vertex with no incoming edges (that is, with in-degree 0). Let v_1 be such a vertex. Indeed, if v_1 did not exist, then in tracing a directed path from an arbitrary start vertex, we would eventually encounter a previously visited vertex, thus contradicting the acyclicity of $\vec{G}$. If we remove v_1 from $\vec{G}$, together with its outgoing edges, the resulting directed graph is still acyclic. Hence, the resulting directed graph also has a vertex with no incoming edges, and we let v_2 be such a vertex. By repeating this process until the directed graph becomes empty, we obtain an ordering $v_1, \dots, v_n$ of the vertices of $\vec{G}$. Because of the construction above, if (v_i, v_j) is an edge of $\vec{G}$, then v_i must be deleted before v_j can be deleted, and thus, $i < j$. Therefore, $v_1, \dots, v_n$ is a topological ordering. ■

Proposition 14.21’s justification suggests an algorithm for computing a topological ordering of a directed graph, which we call **topological sorting**. We present a Java implementation of the technique in Code Fragment 14.11, and an example execution of the algorithm in Figure 14.13. Our implementation uses a map, named `inCount`, to map each vertex v to a counter that represents the current number of incoming edges to v , excluding those coming from vertices that have previously been added to the topological order. As was the case with our graph traversals, a hash-based map only provides $O(1)$ expected time access to its entries, rather than worst-case time. This could easily be converted to worst-case time if vertices could be indexed from 0 to $n - 1$, or if we store the count as a field of the vertex instance.

As a side effect, the topological sorting algorithm of Code Fragment 14.11 also tests whether the given directed graph $\vec{G}$ is acyclic. Indeed, if the algorithm terminates without ordering all the vertices, then the subgraph of the vertices that have not been ordered must contain a directed cycle.

```

1  /** Returns a list of vertices of directed acyclic graph g in topological order. */
2  public static <V,E> PositionalList<Vertex<V>> topologicalSort(Graph<V,E> g) {
3      // list of vertices placed in topological order
4      PositionalList<Vertex<V>> topo = new LinkedPositionalList<>();
5      // container of vertices that have no remaining constraints
6      Stack<Vertex<V>> ready = new LinkedStack<>();
7      // map keeping track of remaining in-degree for each vertex
8      Map<Vertex<V>, Integer> inCount = new ProbeHashMap<>();
9      for (Vertex<V> u : g.vertices()) {
10         inCount.put(u, g.inDegree(u));           // initialize with actual in-degree
11         if (inCount.get(u) == 0)                 // if u has no incoming edges,
12             ready.push(u);                       // it is free of constraints
13     }
14     while (!ready.isEmpty()) {
15         Vertex<V> u = ready.pop();
16         topo.addLast(u);
17         for (Edge<E> e : g.outgoingEdges(u)) { // consider all outgoing neighbors of u
18             Vertex<V> v = g.opposite(u, e);
19             inCount.put(v, inCount.get(v) - 1); // v has one less constraint without u
20             if (inCount.get(v) == 0)
21                 ready.push(v);
22         }
23     }
24     return topo;
25 }

```

Code Fragment 14.11: Java implementation for the topological sorting algorithm. (We show an example execution of this algorithm in Figure 14.13.)

Proposition 14.22: Let $\vec{G}$ be a directed graph with n vertices and m edges, using an adjacency list representation. The topological sorting algorithm runs in $O(n+m)$ time using $O(n)$ auxiliary space, and either computes a topological ordering of $\vec{G}$ or fails to include some vertices, which indicates that $\vec{G}$ has a directed cycle.

Justification: The initial recording of the n in-degrees uses $O(n)$ time based on the `inDegree` method. Say that a vertex u is *visited* by the topological sorting algorithm when u is removed from the ready list. A vertex u can be visited only when `inCount.get(u)` is 0, which implies that all its predecessors (vertices with outgoing edges into u) were previously visited. As a consequence, any vertex that is on a directed cycle will never be visited, and any other vertex will be visited exactly once. The algorithm traverses all the outgoing edges of each visited vertex once, so its running time is proportional to the number of outgoing edges of the visited vertices. In accordance with Proposition 14.9, the running time is $(n+m)$. Regarding the space usage, observe that containers `topo`, `ready`, and `inCount` have at most one entry per vertex, and therefore use $O(n)$ space. ■

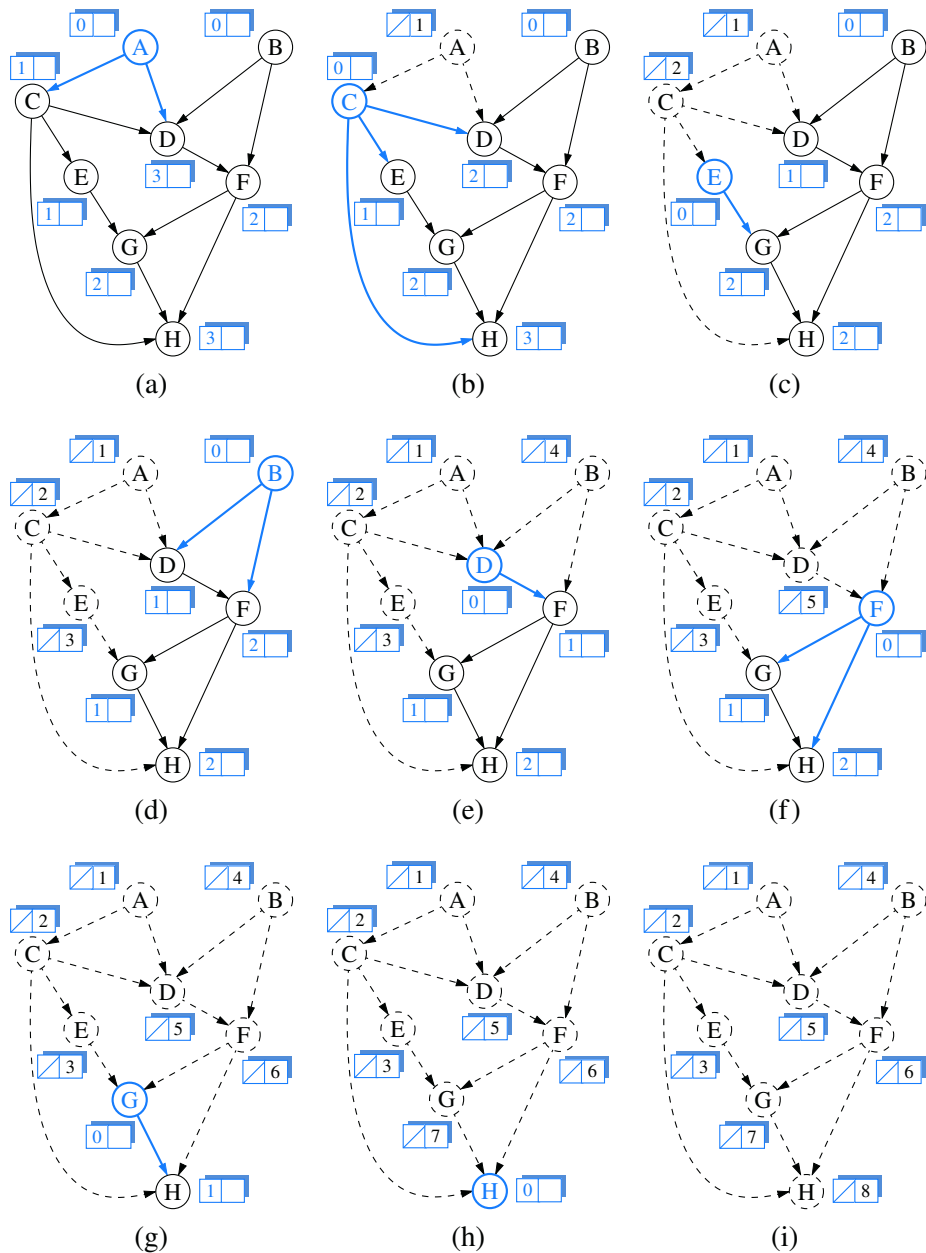


Figure 14.13: Example of a run of algorithm `topologicalSort` (Code Fragment 14.11). The label near a vertex shows its current *inCount* value, and its eventual rank in the resulting topological order. The highlighted vertex is one with *inCount* equal to zero that will become the next vertex in the topological order. Dashed lines denote edges that have already been examined, which are no longer reflected in the *inCount* values.